A Project report on

ROOMIERADAR-SMART ROOMMATE FINDER PORTAL

Submitted in partial fulfillment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

by

K NIVEDITHA	**224G1A0563**
P AJAY KUMAR	**224G1A0502**
A S SABIHA AFRIN	**224G1A0588**

Under the Guidance of

Dr. S. Sunitha, M.Tech., Ph.D.

Assistant Professor

Computer Science & Engineering

SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY

(AUTONOMOUS)

Rotarypuram Village, B K Samudram Mandal, Ananthapuramu - 515701

2025-2026

SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY

(AUTONOMOUS)

(Affiliated to JNTUA, Accredited by NAAC with 'A' Grade, Approved by AICTE, New Delhi & Accredited by NBA (EEE, ECE, CSE, CIVIL & MECHANICAL)
Rotarypuram Village, Bukkarayasamudram Mandal, Ananthapuramu-515701

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Certificate

This is to certify that the Project report entitled **ROOMIERADAR-Smart Roommate Finder Portal** is the Bonafide work carried out by **K Niveditha, P Ajay kumar, A S Sabiha Afrin** bearing Roll Number **224G1A0563, 224G1A0502, 224G1A0588** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science & Engineering** during the academic year 2025 - 2026.

Project Guide
Dr. S. Sunitha, M.Tech., Ph.D.
Assistant Professor

Head of the Department
Dr. P. Veera Prakash, M.Tech., Ph.D.
Assistant Professor

Date:

Place: Rotarypuram

External Examiner

DECLARATION CERTIFICATE

We students of **Computer Science and Engineering**, **SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY(AUTONOMOUS)**, Rotarypuram, hereby declare that the dissertation entitled **"ROOMIERADAR-Smart Roommate Finder Portal"** embodies the report of our project work carried out by us during IV year under the guidance of Dr. S. Sunitha, M.Tech., Ph.D., Assistant Professor, Department of Computer Science and Engineering, Srinivasa Ramanujan Institute of Technology, and this work has been submitted for the partial fulfillment of the requirements for the award of degree of Bachelor of Technology.

The results embodied in this project report have not been submitted to any other University or Institute for the award of any Degree or Diploma.

Date:

Place:

S.No.	Name of the Student	Roll Number	Signature
1	K Niveditha	224G1A0563	
2	P Ajay Kumar	224G1A0502	
3	A S Sabiha Afrin	224G1A0588	

Vision & Mission of the SRIT

Vision:

To become a premier Educational Institution in India offering the best teaching and learning environment for our students that will enable them to become complete individuals with professional competency, human touch, ethical values, service motto, and a strong sense of responsibility towards environment and society at large.

Mission:

- Continually enhance the quality of physical infrastructure and human resources to evolve in to a center of excellence in engineering education.
- Provide comprehensive learning experiences that are conducive for the students to acquire professional competences, ethical values, life-long learning abilities and understanding of the technology, environment and society.
- Strengthen industry institute interactions to enable the students work on realistic problems and acquire the ability to face the ever changing requirements of the industry.
- Continually enhance the quality of the relationship between students and faculty which is a key to the development of an exciting and rewarding learning environment in the college.

Vision & Mission of the Department of CSE

Vision:

To evolve as a leading department by offering best comprehensive teaching and learning practices for students to be self-competent technocrats with professional ethics and social responsibilities.

Mission:

DM 1: Continuous enhancement of the teaching-learning practices to gain profound knowledge in theoretical & practical aspects of computer science applications.

DM 2: Administer training on emerging technologies and motivate the students to inculcate self-learning abilities, ethical values and social consciousness to become competent professionals.

DM 3: Perpetual elevation of Industry-Institute interactions to facilitate the students to work on real-time problems to serve the needs of the society.

Program Educational Objectives (PEOs)

An SRIT graduate in Computer Science & Engineering, after three to four years of graduation will:

PEO 1: Lead a successful professional career in IT / ITES industry / Government organizations with ethical values.

PEO 2: Become competent and responsible computer science professional with good communication skills and leadership qualities to respond and contribute significantly for the benefit of society at large.

PEO 3: Engage in life-long learning, acquiring new and relevant professional competencies / higher academic qualifications.

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of people who made it possible, whose constant guidance and encouragement crowned our efforts with success. It is a pleasant aspect that we have now the opportunity to express our gratitude for all of them.

It is with immense pleasure that we would like to express our indebted gratitude to our Guide **Dr. S. Sunitha, Assistant Professor, Computer Science & Engineering,** who has guided us a lot and encouraged us in every step of the project work. We thank her for the stimulating guidance, constant encouragement and constructive criticism which have made possible to bring out this project work.

We express our deep-felt gratitude to **Mr. K. Lokeshnath, Assistant Professor, and Mr. M. Narasimhulu, Assistant Professor, Project Coordinators,** for their valuable guidance and unstinting encouragement which enabled us to accomplish our project successfully in time.

We are very much thankful to **Dr. P. Veera Prakash, Assistant Professor & Head of the Department, Computer Science & Engineering,** for his kind support and for providing necessary facilities to carry out the work.

We wish to convey our special thanks to **Dr. G. Balakrishna, Principal** of **Srinivasa Ramanujan Institute of Technology,** for giving the required information in doing our project work. Not to forget, we thank all other faculty and non- teaching staff, and our friends who had directly or indirectly helped and supported us in completing our project in time.

We also express our sincere thanks to the Management for providing excellent facilities**.**

Finally, we wish to convey our gratitude to our families who fostered all the requirements and facilities that we need.

Project Associates

224G1A0563

224G1A0502

224G1A0588

ABSTRACT

Finding compatible roommates and suitable shared accommodation is a common challenge for students and young professionals relocating for education or employment. Traditional methods of searching for roommates are often time-consuming and unreliable, frequently leading to mismatched living arrangements and conflicts. To address this issue, this project proposes ROOMIERADAR – Smart Roommate Finder Portal**,** a web-based platform designed to simplify the roommate matching process.

The system allows users to register, create profiles, and specify accommodation preferences such as lifestyle habits, cleanliness level, sleep schedules, and Study habits. A rule-based matching algorithm analyzes these preferences and calculates compatibility scores to recommend suitable roommates. The platform also provides features such as secure user authentication, profile management, and a messaging system that enables users to communicate before confirming shared accommodation.

By providing preference-based matching and a structured digital platform, ROOMIERADAR reduces the time and effort required to find compatible roommates while promoting better shared living experiences.

Keywords: *Roommate Matching System, Shared Accommodation, Rule-Based Matching Algorithm, Preference-Based Recommendation, Compatibility Scoring, Web Application, Django Framework.*

CONTENTS

Page No.

Abstract		**VI**
List of Figures		**IX**
List of Tables		**X**
Abbreviations		**XI**
Chapter 1	**Introduction**	**1-3**
	1.1 Problem Statement	1
	1.2 Objectives	2
	1.3 Scope of the Study	3
	1.4 Significance of the Study	3
Chapter 2	**Literature Survey**	**4-5**
Chapter 3	**Planning**	**6-13**
	3.1 Existing System	6
	3.1.1 Disadvantages	7
	3.2 Proposed System	8
	3.2.1 Advantages	8
	3.3 Agile project planning	8
	3.3.1 Agile approach	9
	3.3.2 Scrum framework	9
	3.3.3 Sprint planning & Backlog	10
	3.3.4 Product Backlog	10
	3.4 Objectives of Planning	11
	3.4.1 Performance	11
	3.4.2 Cost	12
	3.4.3 Time	12
	3.5 System Modules and Functionalities	12
	3.5.1 User Authentication Module	12
	3.5.2 User Profile Management Module	12
	3.5.3 Preference Configuration Module	12
	3.5.4 Roommate Matching Module	13
	3.5.5 Room Booking Module	13
	3.5.6 Messaging Module	13
	3.5.7 Admin Management Module	13
Chapter 4	**Agile Methodology & Development Process**	**14-22**
	4.1 Research Methodology	14
	4.2 Agile Methodology Implementation	14
	4.2.1 Agile Workflow & Iterations	14
	4.2.2 Sprint Reviews & Retrospectives	14
	4.3 System Design	15
	4.4 System Development & Sprint execution	20
	4.4.1 Sprint wise feature implementation	20
	4.4.2 Incremental Deliverables	20

	4.5 Testing and Evaluation	20
	4.6 System Architecture	21
	4.7 System Components and flow	22
Chapter 5	**Implementation**	**23-31**
	5.1 Experimental Setup	23
	5.1.1 Hardware Requirements	23
	5.1.2 Software Requirements	23
	5.2 Technologies Used	23
	5.3 Source Code	25
	5.3.1 Main Application file (Manage.py)	25
	5.3.2 Project Configuration File(Settings.py)	25
	5.3.3 Home module(home)	26
	5.3.4 Accounts Module(accounts)	26
	5.3.5 Base Module(base)	27
	5.3.6 Chat Module(chat)	28
	5.3.7 Room and Booking Module (roomieradar_app)	29
	5.3.8 Admin Module (admin)	30
	5.3.9 URL Routing Configuration (urls.py)	31
Chapter 6	**Agile Sprint Results & Discussion**	**32-44**
	6.1 Sprint-wise Feature Completion	32
	6.2 Feedback & Improvements	33
	6.3 Performance analysis and KPIs	34
Chapter 7	**Installation Guide**	**34-39**
Chapter 8	**Results**	**40-48**
	Conclusion & Future Enhancements	**49-52**
	References	**53**
	Publications	**55**

List of Figures

Fig. No.	Description	Page No.
4.3.1	Class Diagram Admin	15
4.3.2	User Class Diagram	16
4.3.3	Sequence Diagram (System Backend)	16
4.3.4	User Registration Sequence Diagram	17
4.3.5	Activity Diagram – User Login to Room Booking	18
4.3.6	Basic Flow Diagram	19
4.3.7	Deployment Diagram	19
6.1	Performance Analysis	36
8.1	Home Page	41
8.2	Registration Page	42
8.3	Account Activation Email	42
8.4	Login Page	43
8.5	Main User Dashboard	43
8.6	Roommate Booking Dashboard	44
8.7	Only Hostel Booking	44
8.8	Preferences Page	45
8.9	Roommate Matches	45
8.10	User Profile	46
8.11	My Bookings	46
8.12	chats interface	47
8.13	chat interface	47
8.14	Admin Dashboard	47

List of Tables

Table No.	Title	Page No.
3.1	Existing System disadvantages	6
3.2	Proposed System Advantages	8
3.3	Product Backlog	10
4.1	Sprint-wise features	20
5.1	Hardware Requirements	23
5.2	Software Requirements	23

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
UI	User Interface
SMTP	Simple Mail Transfer Protocol
ORM	Object Relational Mapping
SQL	Structured Query Language
KPI	Key Performance Indicators

CHAPTER 1
INTRODUCTION

In recent years, the demand for shared accommodation has increased significantly as many students and working professionals relocate to new cities for education and employment opportunities. Finding a suitable roommate is often a challenging task because individuals must consider several compatibility factors such as lifestyle habits, cleanliness preferences, daily routines, and personal interests. Traditional methods of searching for roommates through friends, advertisements, or social media platforms are often inefficient and unreliable, which can lead to mismatched living arrangements and conflicts between roommates.

With the advancement of digital technologies, web-based platforms have emerged as effective solutions for connecting individuals with similar needs and preferences. However, many existing accommodation platforms mainly focus on listing rooms rather than helping users find compatible roommates. As a result, individuals may still experience difficulties in identifying roommates whose habits and lifestyles align with their own preferences.

To address these challenges, this project proposes ROOMIERADAR – Smart Roommate Finder Portal**,** a web-based system designed to simplify the process of finding compatible roommates. The platform allows users to create profiles and specify personal preferences such as lifestyle habits, cleanliness level, sleep schedules, and other daily living preferences. The system uses a rule-based matching algorithm to analyze these preferences and determine compatibility between users.

The proposed system aims to provide a reliable and user-friendly platform that improves the roommate search process by promoting compatibility and reducing potential conflicts. By leveraging modern web technologies and structured preference-based matching techniques, ROOMIERADAR offers an efficient digital solution for individuals seeking suitable roommates for shared living arrangements.

1.1 Problem Statement

Finding a compatible roommate is often a difficult and time-consuming task for students and working professionals who move to new places for education or

employment. In many cases, individuals rely on personal contacts, social media groups, or informal networks to search for roommates. These traditional methods do not guarantee compatibility between roommates, which can result in misunderstandings, lifestyle conflicts, and uncomfortable living environments.

Many existing accommodation platforms mainly focus on listing available rooms rather than helping individuals find roommates with similar preferences and living habits. As a result, users may end up sharing living spaces with people whose daily routines, cleanliness standards, or lifestyle choices differ significantly from their own. This lack of compatibility can lead to dissatisfaction and conflicts in shared living arrangements.

Therefore, there is a need for a structured digital platform that helps individuals find suitable roommates based on compatibility factors such as lifestyle habits, personal preferences, and daily routines. A smart roommate matching system can simplify the search process and improve the overall shared living experience by connecting individuals with similar preferences.

1.2 Objectives

To accomplish the project's purpose, the following specific objectives have been established.

i. To develop a web-based platform that helps users find compatible roommates based on their personal preferences and lifestyle habits.
ii. To implement a rule-based matching algorithm that analyzes user profiles and determines compatibility between potential roommates.
iii. To provide a user-friendly interface that allows users to create profiles, manage their preferences, and search for suitable roommates easily.
iv. To enable communication between potential roommates through a messaging system before confirming shared accommodation.
v. To ensure secure user authentication and proper data management to protect user information and maintain privacy within the platform.

1.3 Scope of the Study

This This study focuses on the development of ROOMIERADAR – Smart Roommate Finder Portal, a web-based platform designed to help students within an institution find compatible roommates based on their preferences and lifestyle habits. The scope of the study includes:

- Development of a platform for students to create and manage roommate profiles.
- Implementation of a rule-based system to match compatible roommates.
- Providing communication between potential roommates through the portal.
- Ensuring secure user authentication and profile management.

1.4 Significance of the Study

This study is significant because it:

- Helps students find compatible roommates based on lifestyle preferences and habits.
- Reduces conflicts in shared living by improving compatibility between roommates.
- Saves time and effort by providing a structured digital platform for roommate search.
- Provides a secure and user-friendly system for managing roommate profiles and communication.

CHAPTER 2

LITERATURE SURVEY

ROOMIERADAR – Smart Roommate Finder Portal is designed to simplify the process of finding compatible roommates for students and professionals through a web-based platform. The system allows users to create profiles and specify their preferences to identify suitable roommates. By providing a structured interface and preference-based matching mechanism, the platform improves the efficiency of the roommate search process and enhances shared living experiences.

Banode et al., 2025 proposed a compatibility-based roommate assignment system that evaluates user preferences such as lifestyle habits and behavioral patterns. Their approach demonstrated that automated roommate matching systems can improve compatibility and reduce conflicts in shared accommodation environments **[1]**.

Shahu et al., 2024 developed a smart roommate matching platform that enables users to create profiles and search for suitable roommates using preference-based filtering techniques. Their system improved the efficiency of the roommate search process through structured digital matching **[2]**.

Ighave et al., 2024 introduced a roommate allocation system that utilizes intelligent algorithms to analyze user attributes and lifestyle preferences. The system enhances roommate compatibility by recommending users with similar living habits and interests **[3]**.

Jose et al., 2024 analyzed personality-based matching techniques used in roommate and social matching platforms. Their study highlighted that personality traits and behavioral compatibility significantly influence successful roommate matching **[4]**.

Smith et al., 2024 developed a web-based roommate search platform where users can create profiles and interact with potential roommates. Their study demonstrated that web applications provide a convenient and organized method for finding shared accommodation **[5]**.

Patel et al., 2023 proposed a preference-based recommendation system that analyzes user preferences to suggest compatible roommates. Their approach reduced mismatches in shared accommodation by using structured preference analysis **[6]**.

Garcia et al., 2024 explored graph-based and similarity-based techniques for roommate matching. Their research showed that similarity measurement algorithms can improve the accuracy of compatibility prediction between roommates **[7]**.

Kumar et al., 2020 introduced a web-based roommate finder system that connects individuals seeking shared accommodation through profile-based matching and communication features. Their study emphasized that online platforms significantly simplify the roommate search process **[8]**.

This literature survey highlights the importance of compatibility analysis, preference-based matching techniques, and web-based platforms in improving roommate matching systems. By leveraging these concepts, the proposed ROOMIERADAR – Smart Roommate Finder Portal implements a rule-based matching mechanism to recommend compatible roommates within an institutional environment.

CHAPTER 3

PLANNING

3.1 Existing System

Finding suitable roommates in institutions or shared living environments is often done through traditional and informal methods. These approaches usually lack structured compatibility evaluation and often result in mismatched roommate assignments. The most common existing methods include:

• Manual Roommate Search:

Students often rely on personal contacts, friends, or social media groups to find roommates. This process is unorganized and may not ensure compatibility between individuals.

• Random or Administrative Allocation:

In many hostels or institutions, roommates are assigned randomly or by administrative staff without considering personal preferences or lifestyle habits.

• Social Media and Online Groups:

Some individuals use social media platforms or messaging groups to search for roommates. However, these platforms lack structured profile matching and compatibility analysis.

• Basic Accommodation Listing Platforms:

Some websites provide accommodation listings where users can find rooms or apartments. These platforms mainly focus on property listings rather than matching compatible roommates.

• Lack of Compatibility Analysis:

Most existing systems do not analyze lifestyle preferences, daily habits, or personality traits, which can lead to conflicts between roommates.

3.1.1 Disadvantages

Table 3.1: Existing System Disadvantages

Challenges	Description
Lack of Compatibility Matching	Traditional methods do not analyze lifestyle preferences or habits between roommates.
Random Roommate Allocation	Roommates may be assigned randomly without considering compatibility.

Lack of Structured Platforms	Existing methods rely on informal searches such as social media or personal contacts.
Communication Limitations	Potential roommates may not have a structured way to communicate before sharing accommodation.
Time-Consuming Process	Finding suitable roommates through traditional methods requires significant time and effort.
Higher Risk of Conflicts	Differences in habits and lifestyle may lead to misunderstandings between roommates.

3.2 Proposed System

The proposed system, **ROOMIERADAR – Smart Roommate Finder Portal**, is a web-based platform developed using the **Django framework** and modern web technologies such as **HTML, CSS, and JavaScript**. The system allows students to create profiles, specify their lifestyle preferences, and find compatible roommates within an institutional environment.

The platform uses a **rule-based matching algorithm** to analyze user attributes such as lifestyle habits, cleanliness preferences, and daily routines to determine compatibility between users. Based on these preferences, the system recommends suitable roommate matches, helping users make informed decisions before sharing accommodation

Key Features:

- **User Authentication:** Users must register and log in to access the roommate matching platform securely.
- **Profile Creation:** Users can create detailed profiles including lifestyle preferences, habits, and personal information required for roommate matching.
- **Preference-Based Matching:** The system uses a rule-based matching mechanism to identify compatible roommates based on user preferences.
- **Roommate Discovery:** Users can explore potential roommate profiles and find matches that align with their lifestyle and habits.

- **Messaging Feature:** The platform allows users to communicate with potential roommates before confirming shared accommodation.
- **User-Friendly Interface:** The application provides an intuitive interface that makes it easy for users to search, view profiles, and connect with compatible roommates.

3.2.1 Advantages

Table 3.2: Proposed System Advantages

Advantage	Description
Compatibility-Based Matching	The system matches roommates based on lifestyle preferences and personal habits.
Structured Digital Platform	Provides an organized platform for students to search and connect with roommates.
Easy Communication	Enables users to communicate with potential roommates through the platform.
Time Efficiency	Reduces the time required to find suitable roommates.
Improved Living Experience	Helps minimize conflicts by matching users with compatible roommates.
Secure User Access	The system provides authentication to ensure that only registered users can access the platform.
Simple and Efficient Rule based Algorithm	The rule-based approach is easy to implement, understand, and maintain without requiring large datasets.

3.3 Agile Project Planning

The development of **ROOMIERADAR – Smart Roommate Finder Portal** follows the **Agile software development methodology** to ensure flexibility and continuous improvement during the development process. Agile allows the project to be developed in small iterations called sprints, enabling regular testing and feedback. This approach helps the development team adapt to changing requirements and deliver functional modules efficiently. The following sections describe the Agile approach, Scrum framework, sprint planning, and product backlog used in the project.

3.3.1 Agile Approach

The Agile approach follows an **iterative and incremental development methodology**, enabling continuous collaboration between the development team and stakeholders. This approach helps in delivering system features step by step while allowing improvements during each development cycle. It ensures:

- Faster development and delivery of functional modules.
- Flexibility to adapt to changing requirements during development.
- Continuous testing and integration of feedback.
- Improved system quality through regular updates and refinements.

By applying Agile principles, the **ROOMIERADAR – Smart Roommate Finder Portal** can be developed efficiently while accommodating evolving requirements and ensuring a reliable and user-friendly platform.

3.3.2 Scrum Framework

The project adopts the **Scrum framework**, which is a structured Agile methodology used to organize development activities into manageable iterations called sprints. Scrum helps in improving collaboration, monitoring progress, and delivering functional components of the system efficiently. The framework consists of the following elements:

Roles:

1. **Product Owner:** Defines the product vision and manages the backlog.
2. **Scrum Master:** Ensures Agile practices are followed and resolves issues.
3. **Development Team:** Developers and testers responsible for building and testing system modules.

Scrum Artifacts:

1. **Product Backlog:** List of all system features and requirements.
2. **Sprint Backlog:** Tasks selected for completion in a sprint.
3. **Increment:** Working version of the system after each sprint.

Scrum Events:

1. **Sprint Planning:** Selecting tasks for the sprint.
2. **Daily Standup:** Short meeting to track progress.

3. **Sprint Review:** Demonstration of completed work.
4. **Sprint Retrospective:** Evaluation and improvement of the sprint process.

This framework ensures effective collaboration and continuous development of the system.

3.3.3 Sprint Planning & Backlog

Sprint planning defines the tasks and deliverables for each development cycle. The ROOMIERADAR system is developed using a **16-week timeline**, divided into **2-week sprints**, where each sprint focuses on implementing specific system features.

Sprint Duration: *2 Weeks per Sprint*

Total Sprints: *8 Sprints*

Sprint Structure:

- **Sprint 1:** Requirement analysis, system design, and UI planning.
- **Sprint 2:** Database setup and backend development using Django.
- **Sprint 3:** User registration and authentication module.
- **Sprint 4:** Profile creation and preference management.
- **Sprint 5:** Rule-based roommate matching algorithm implementation.
- **Sprint 6:** Messaging feature between users.
- **Sprint 7:** System testing, debugging, and performance improvement.
- **Sprint 8:** Deployment, documentation, and final improvements.

3.3.4 Product Backlog

The **product backlog** contains a list of all the **features, enhancements, and technical requirements** that will be implemented throughout the project.

Table 3.3: Product Backlog

Backlog Item	Description	Priority
User Registration & Authentication	Allow users to register and log in securely with email verification to prevent unauthorized access.	High
Preference Configuration	Allow users to specify lifestyle habits and compatibility preferences for roommate matching.	High
Rule-Based Matching Algorithm	Match users based on preferences and display compatible roommate profiles.	High
Room Booking System	Allow users to book rooms once compatible roommate matches are found.	High

Messaging System	Enable communication between potential roommates before confirming room booking.	Medium
User Profile Creation	Allow users to create and manage roommate profiles with personal details and lifestyle information.	Medium
Email Notification System	Send verification emails for account activation and booking confirmation.	Medium
Chatbot Assistance	Provide automated responses to user queries related to the website and its features.	Low

The product backlog is continuously refined based on stakeholder feedback and sprint reviews to ensure efficient system development.

This Agile project planning ensures that the **ROOMIERADAR – Smart Roommate Finder Portal** is developed in an iterative, efficient, and user-centric manner while incorporating feedback and improvements at each stage of development.

3.4 Objectives of Planning

The planning phase for the **ROOMIERADAR – Smart Roommate Finder Portal** focuses on defining the system scope, selecting appropriate technologies, and organizing the development process. The main objectives of planning include:

- **Defining System Scope:** Identify key features such as user authentication, profile creation, roommate matching, and messaging.
- **Technology Selection:** Use technologies such as the **Django framework, HTML, CSS, JavaScript, and SQLite database** for developing the web application.
- **System Architecture Design:** Plan the integration of frontend, backend, database, and the rule-based matching algorithm.
- **Feasibility Analysis:** Evaluate the technical and operational feasibility of implementing the roommate matching system.
- **Risk Identification:** Identify possible risks such as data security issues, incorrect user data, and system misuse.

These objectives ensure the structured and efficient development of the **ROOMIERADAR system**.

3.4.1 Performance

The **ROOMIERADAR – Smart Roommate Finder Portal** improves the

roommate search process by using a rule-based matching algorithm to identify compatible roommates based on user preferences. The system efficiently processes user profiles and preferences to generate suitable roommate matches.

The platform ensures reliable performance through **secure authentication, efficient database operations using Django ORM with SQLite, and real-time communication using WebSockets (Django Channels)** for messaging between users. Email verification and account activation are handled through **SMTP-based email services**, ensuring secure user authentication.

Since the application operates as a **self-contained Django system without external API dependencies**, all data processing, matching logic, and communication are managed internally, providing stable performance and quick response for user interactions.

3.4.2 Cost

- Time: 4 Months
- Average Staff Required: 3 Persons

3.4.3 Time

The project is estimated to take **4 months (12 person-months)**, including system design, development, testing, and deployment. The planned timeline ensures efficient implementation while maintaining system quality and functionality.

3.5 System Modules and Functionalities

The **ROOMIERADAR – Smart Roommate Finder Portal** consists of several modules that handle different functionalities of the system.

3.5.1 User Authentication Module

- User registration with email verification
- Secure login and logout functionality

3.5.2 User Profile Management Module

- Profile creation and editing
- User information management

3.5.3 Preference Configuration Module

- Setting lifestyle preferences and habit
- Room sharing preferences (single, 2-sharing, 3-sharing, 4-sharing)
- Compatibility factors such as bedtime, cleanliness, and food type

3.5.4 Roommate Matching Module

- Rule-based compatibility matching
- Calculation of similarity scores
- Display of compatible roommate profiles

3.5.5 Room Booking Module

- Viewing available rooms and beds
- Booking rooms after successful matching
- Managing booking details

3.5.6 Messaging Module

- Real-time messaging between users
- Communication before confirming room booking

3.5.7 Admin Management Module

- Admin dashboard for system monitoring
- User and room management
- Booking oversight

CHAPTER 4
AGILE METHODOLOGIES & DEVELOPMENT PROCESS

The development of ROOMIERADAR, a Smart Roommate Finder Portal, employs an Agile Software Development Lifecycle to enhance flexibility and efficiency through iterative development. Utilizing the Django framework and SQLite database, the platform incorporates WebSockets for real-time messaging and SMTP for account verification. It offers a web-based solution that allows users to create profiles, set preferences, find compatible roommates via a matching algorithm, communicate through messaging, and book rooms post-matching.

4.1 Research Methodology

The development of **ROOMIERADAR – Smart Roommate Finder Portal** is based on analyzing existing roommate search challenges, technological feasibility, and user requirements.

- **Primary Research** – Understanding the challenges students face while finding compatible roommates and shared accommodation.
- **Secondary Research** – Studying existing roommate matching systems and accommodation platforms to identify limitations and improvements.
- **Technical Research** – Analyzing suitable web technologies including **Django, HTML, CSS, JavaScript, SQLite, and WebSockets** to develop a secure, scalable, and efficient roommate matching platform with real-time communication features.

4.2 Agile Methodology Implementation

The The project employs the Agile development methodology, enabling the system to be developed through small iterations known as sprints, which fosters continuous improvement, flexibility, and swift adaptation to changing requirements.

4.2.1 Agile Workflow & Iterations

• The development process is divided into multiple **sprints**, each focusing on implementing specific features of the system.
• User stories define system functionalities and guide development tasks.
• Continuous integration and testing ensure system stability and reliability after each sprint.

4.2.2 Sprint Reviews & Retrospectives

- **Sprint Reviews** – Demonstrate the completed system features at the end of each sprint.
- **Sprint Retrospectives** – valuate the development process and identify

improvements for future sprints.

4.3 System Design

- **Architecture Design –** Defines the system structure including frontend, backend (Django), and SQLite database.
- **UI/UX Design –** Ensures a simple and user-friendly interface for profile creation, preference setup, and roommate matching.
- **Data Flow Diagrams (DFDs) –** Show how data moves through the system between users, processes, and the database.
- **Use Case Diagram –** Illustrates interactions between users and the system such as registration, profile creation, matching, messaging, and room booking.
- **Class Diagram** – Defines the main system components and their relationships (e.g., User, Profile, Preferences, Room, Booking, Message).

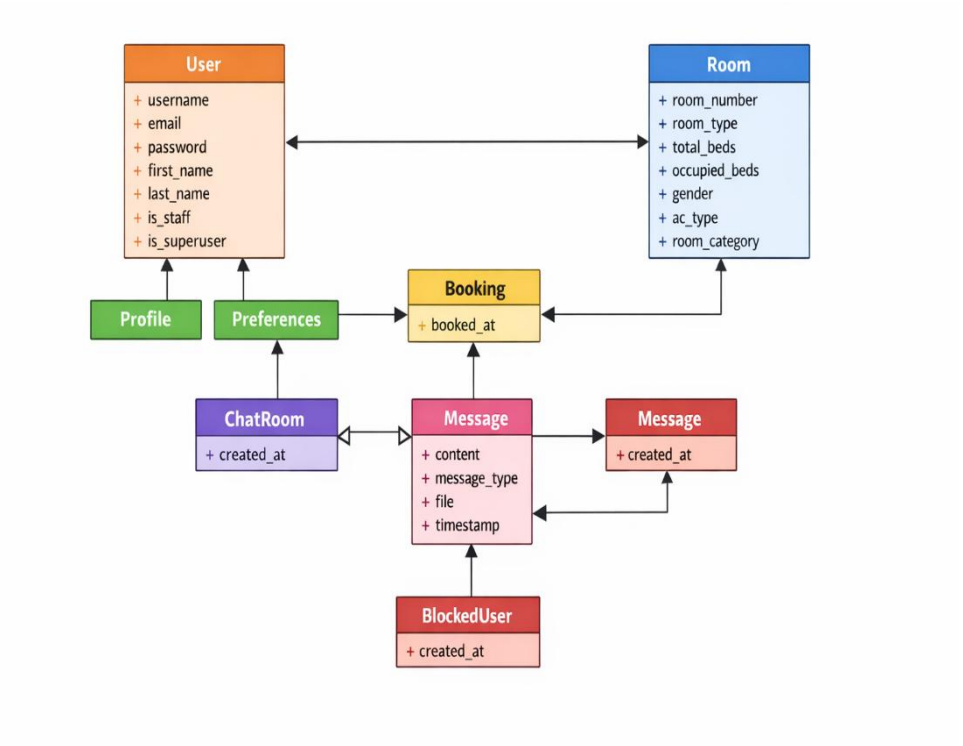


fig 4.3.1: Class Diagram Admin

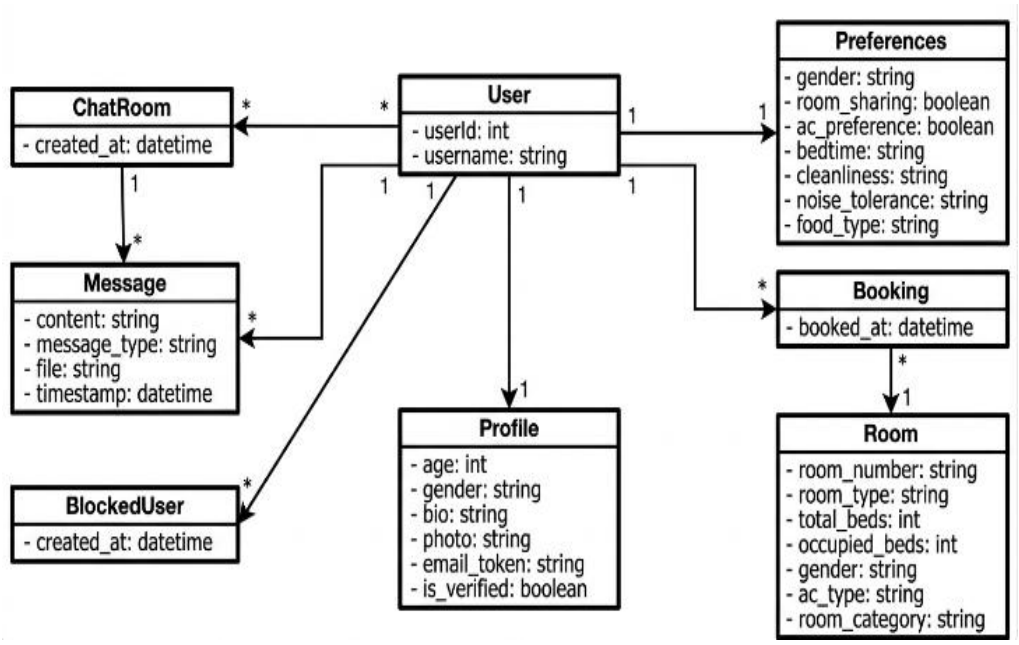


fig 4.3.2: User Class Diagram

- **Sequence Diagram** – Shows the sequence of interactions between users and the system during operations such as user registration, profile creation, roommate matching, messaging, and room booking.

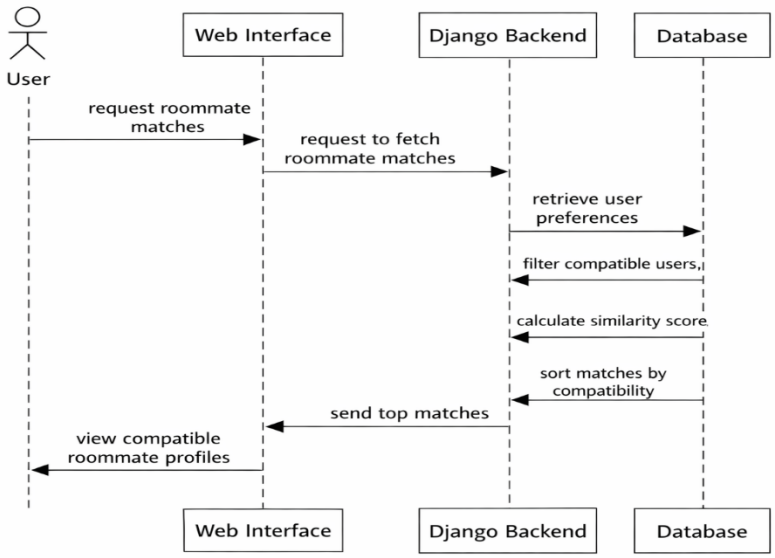


fig 4.3.3: Sequence Diagram (System Backend)

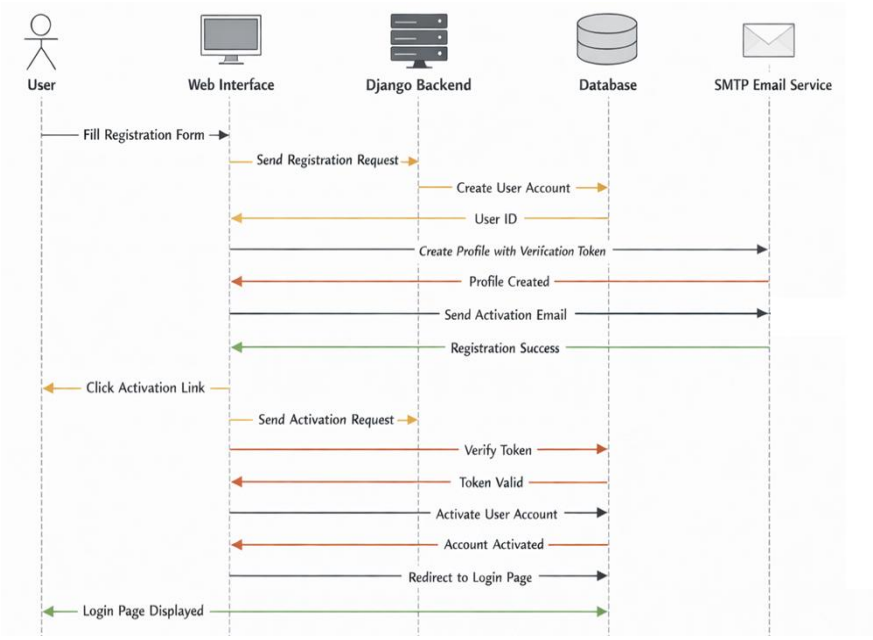


fig 4.3.4: User Registration Sequence Diagram

- **Activity Diagram –** Illustrates the step-by-step workflow of the system, from user login and preference setup to roommate matching and room booking.

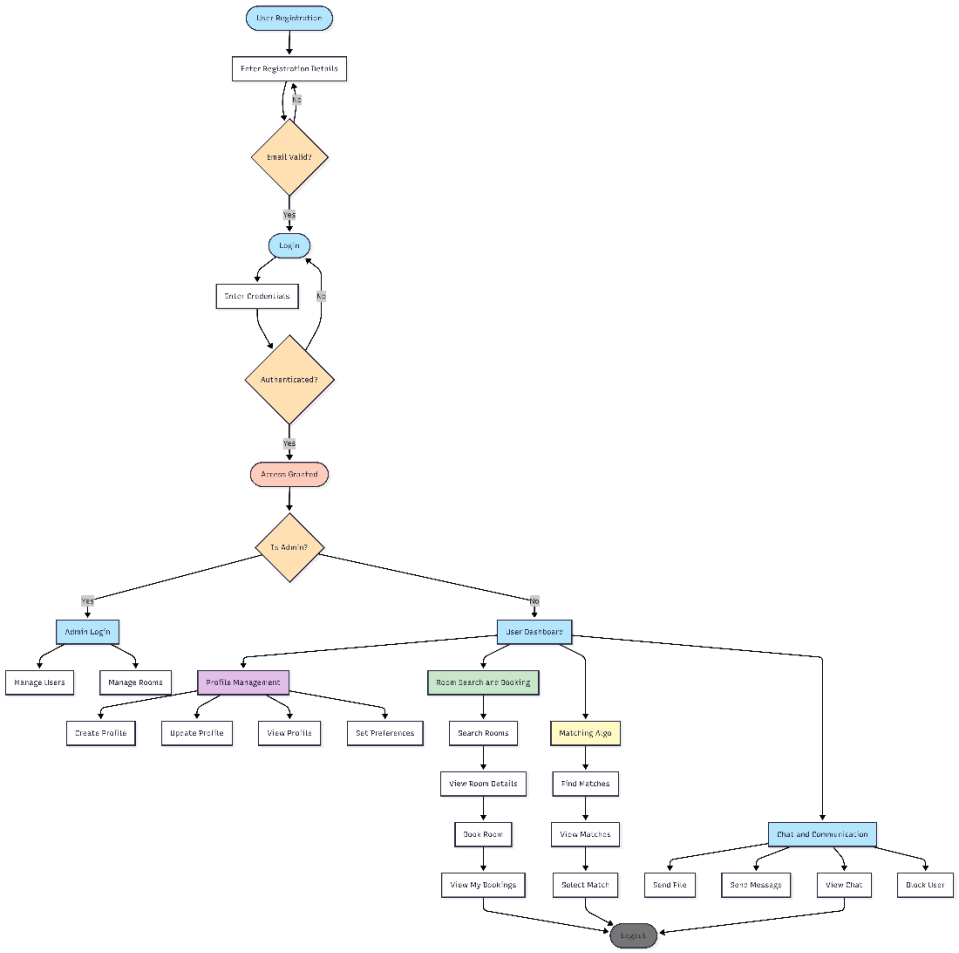


fig 4.3.5: Activity Diagram – User Login to Room Booking

- **Flowchart** – Explains the decision-making process in the ROOMIERADAR system, such as user authentication, roommate matching, and room booking.

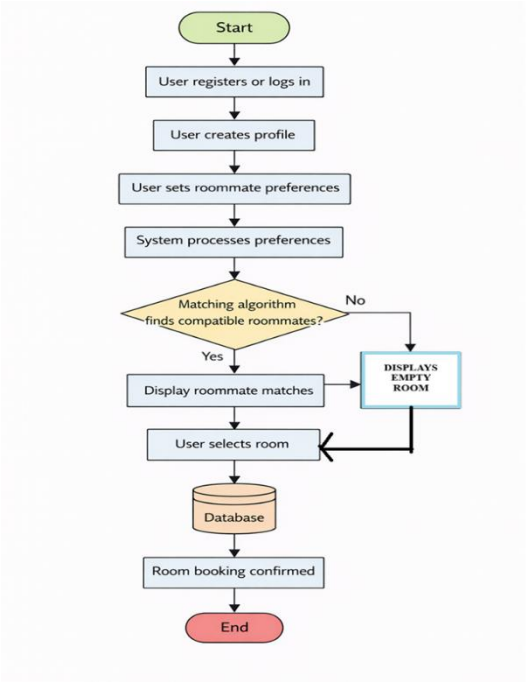


fig 4.3.6: Basic Flow Diagram

- **Deployment Diagram** – Shows how the ROOMIERADAR system is deployed across different components such as the user device, Django web server, SQLite database, and email service.

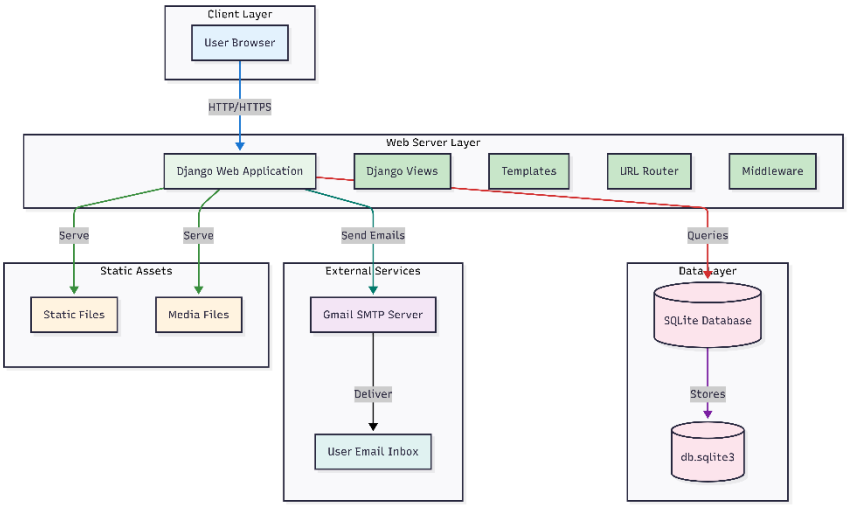


fig 4.3.7: Deployment Diagram

4.4 System Development & Sprint Execution

The development of ROOMIERADAR – Smart Roommate Finder Portal follows an Agile sprint-based approach. The project is divided into multiple sprints where each sprint focuses on implementing specific features of the system.

4.4.1 Sprint-wise Feature Implementation

Table 4.1: Sprint-wise Feature

Sprint No	Features Implemented
Sprint 1	Requirement analysis, system design, and UI planning
Sprint 2	Database setup and backend development using Django
Sprint 3	User registration and authentication module
Sprint 4	Profile creation and preference management
Sprint 5	Rule-based roommate matching algorithm implementation
Sprint 6	Messaging feature between users
Sprint 7	System testing, debugging, and performance improvement
Sprint 8	Deployment, documentation, and final improvements

4.4.2 Incremental Deliverables

- Each sprint delivers a functional module that integrates with the overall system.
- Continuous feedback helps improve system functionality and performance.

4.5 Testing and Evaluation

- **Unit Testing** – Individual modules such as authentication, matching, and booking are tested separately.
- **Integration Testing** – Ensures proper interaction between the frontend, backend, and database.
- **User Acceptance Testing (UAT)** – Validates system usability and functionality from the user perspective.
- **Performance Testing** – Evaluates system responsiveness and stability during multiple user interactions.

By implementing Agile methodology, the ROOMIERADAR – Smart Roommate Finder Portal ensures a structured, user-centric, and adaptable development process, leading to a reliable and efficient roommate matching platform.

4.6 System Architecture

The **ROOMIERADAR system** is a web-based application developed using the **Django framework with SQLite database**. The architecture consists of the following components:

1. User Interface (UI)

- Built using **HTML, CSS, and JavaScript**.
- Allows users to register, log in, set preferences, and find roommates.

2. Backend Server

- Developed using the **Django framework (Python)**.
- Handles authentication, matching algorithm, messaging, and booking logic.

3. Database

- **SQLite database** stores user profiles, preferences, rooms, bookings, and messages.
- Ensures efficient data storage and retrieval for system operations

4. Matching System

- Implements a rule-based matching algorithm to identify compatible roommates.
- Compares user preferences and calculates similarity scores.

5. Messaging System

- Allows users to communicate with potential roommates through the chat module.
- Supports message exchange before confirming room booking.

6. Email Authentication

- Uses SMTP email service to send account verification emails.
- Ensures that only verified users can access the platform.

7. Security and Access Control

- Provides secure login using Django authentication.
- Manages user sessions and prevents unauthorized access.

4.7 System Components and Flow

1. **User Registration & Authentication**
   - Users register on the platform using their email and create login credentials.
   - Email verification is required to activate the account and access the system.
2. **Profile Creation and Preference Setup**
   - Users create profiles and provide personal details required for roommate matching.
   - Users set lifestyle preferences such as room sharing, cleanliness, bedtime, and food habits.
3. **Roommate Matching**
   - The system processes user preferences using a rule-based matching algorithm.
   - Compatible roommates are displayed based on similarity scores.
4. **Messaging Between Users**
   - Users can communicate with potential roommates through the chat feature.
   - Messaging helps users discuss accommodation before confirming a room.
5. **Room Booking**
   - Users can view available rooms and select a suitable option.
   - The system confirms the booking and updates room availability.

CHAPTER 5

IMPLEMENTATION

5.1 Experimental Setup

The **ROOMIERADAR – Smart Roommate Finder Portal** is developed using the **Django framework with SQLite database**. The system is implemented as a web-based application that allows users to register, create profiles, set preferences, find compatible roommates, communicate through messaging, and book rooms.

5.1.1 Hardware Requirements

The system requires the following hardware for development and testing:

Table 5.1: Hardware Requirements

Components	Specifications
Development Machine	Minimum: Intel i5 Processor, 8GB RAM, 256GB SSD
Server	Localhost / Web server running Django
Network Connectivity	Stable internet connection
Mobile Devices	Android/iOS devices for responsive testing

5.1.2 Software Requirements

The system is built using modern web development technologies and tools:

Table 5.2: Software Requirements

Software	Purpose
Operating System	Windows 10/11, macOS, or Linux
Backend Framework	Django (Python)
Frontend Development	HTML, CSS, JavaScript
Database Management	SQLite
Email Service	SMTP (Gmail)
Version Control	Git / Github
Code Editor	PyCharm IDE

5.2 Technologies Used:

1. Frontend

- Technologies: HTML, CSS, JavaScript
- Purpose: Provides the user interface for interacting with the system.

Features:

- Responsive web design

- Form handling for user input
- Dynamic interaction with the backend

2. Backend

- Technology: Django (Python Framework)
- Purpose: Handles server-side logic and application functionality.

Features:

- User authentication and session management
- Rule-based roommate matching algorithm
- Messaging and room booking logic

3. Database

- Technology: SQLite
- Purpose: Stores system data including users, profiles, preferences, rooms, bookings, and messages.

Features:

- Lightweight database management
- Fast data retrieval and storage
- Integrated with Django ORM

4. Email Authentication System

- Technology: SMTP Email Service
- Purpose: Sends account verification emails for secure user authentication.

Features:

- Email verification during registration
- Secure account activation process

5. Real-Time Messaging System

- Technology: WebSockets (Django Channels)
- Purpose: Enables real-time communication between users in the chat module.

Features:

- Supports instant message exchange between roommates.
- Maintains persistent connection between client and server for live chat updates

6. Additional Tools

- GitHub: Version control and collaborative development.
- Figma: UI/UX desig

5.3 Source Code

5.3.1 Main Application File (manage.py)

The **manage.py** file is the main entry point of the ROOMIERADAR Django application. It is used to perform administrative tasks such as running the development server, managing database migrations, and executing various project commands.

```
RoomieRadar > manage.py
#!/usr/bin/env python
"""Django's command-line utility for administrative tasks."""
import os
import sys


def main():
    """Run administrative tasks."""
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'roomieradar.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)


if __name__ == '__main__':
    main()
```

Explanation of Code:

The manage.py file acts as the main command-line entry point for the ROOMIERADAR Django application. It initializes the Django environment by specifying the project's settings module and then executes administrative commands through Django's built-in management utility. This file allows developers to run important project operations such as starting the development server, creating database migrations, and managing the application. It also includes error handling to ensure that Django is properly installed and configured before executing any commands.

5.3.2 Project Configuration File (settings.py)

```
INSTALLED_APPS = [
    'jazzmin',

    # Django default apps
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # Project apps
    'accounts',
    'base',
    'chat',
    'roomieradar_app',
    'home',
]
```

Key Functionalities:

- INSTALLED_APPS**:** Lists all Django applications used in the project.
- DATABASES**:** Configures the SQLite database used for storing system data.

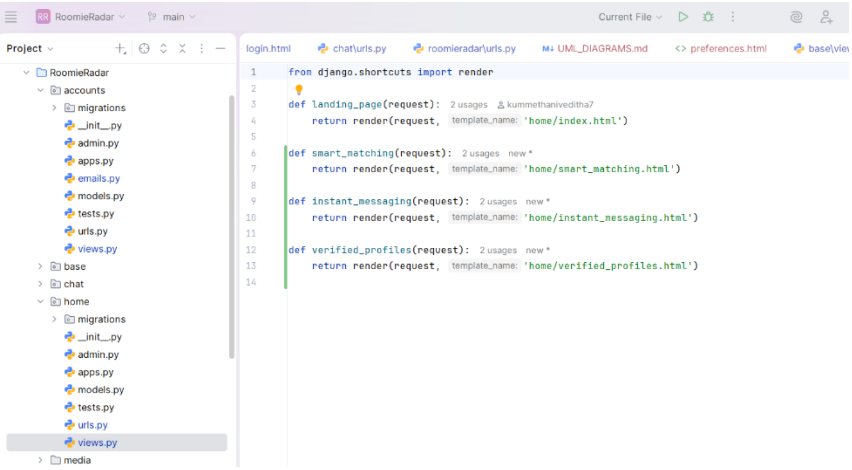
- MIDDLEWARE: Handles request processing, authentication, and security.

How it Works

The settings.py file manages the core configuration of the ROOMIERADAR Django project. It defines installed applications, database connections, middleware components, and system settings that control how the application runs and interacts with different modules.

5.3.3 Home Module (home)

The Home module provides the landing interface of the ROOMIERADAR platform. It displays introductory information about the system and its key features before users register or log in. This module acts as the initial access point for visitors to understand the platform.



Key Functionalities:

- Display landing page with system overview.
- Present platform features and benefits to visitors.
- Provide navigation options for registration and login.
- Highlight system capabilities such as smart matching and messaging.

How it Works

The Home module operates as a presentation layer using Django's template rendering system. When users access the root URL, the corresponding view function loads the landing page template and displays static content describing the system. Since it does not require authentication or database queries, it primarily serves informational content through HTML templates rendered by Django views.

5.3.4 Accounts Module (accounts)

The Accounts module manages user authentication and account operations in the ROOMIERADAR system. It supports user registration, login validation, logout

functionality, and email-based account activation. This module ensures secure user access to the platform.

Key Functionalities:

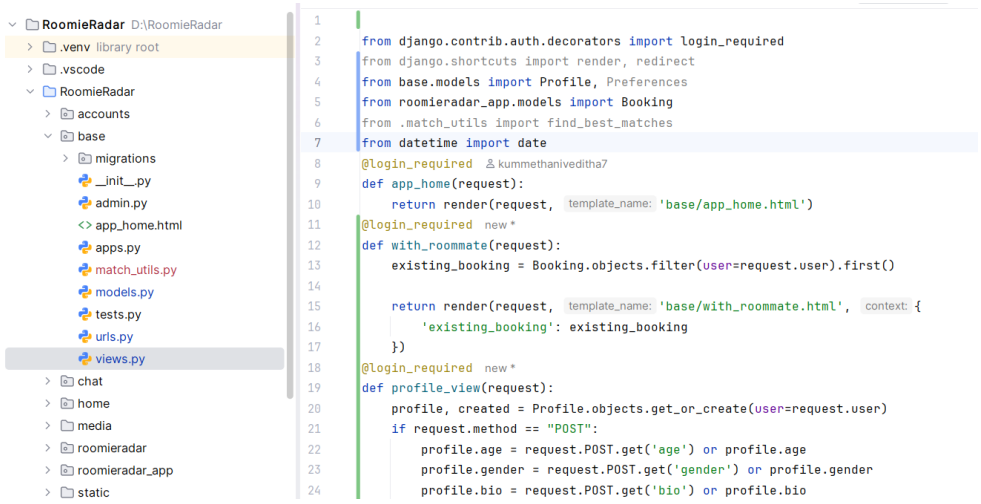
- User registration with email verification.
- Secure login authentication and logout.
- Account activation using token-based verification.
- Session management for authenticated users.

How it Works

The Accounts module integrates Django's built-in authentication framework with custom email verification logic. During registration, a user account is created with an inactive status and a unique verification token is generated. The system sends an activation email using SMTP services. When the user verifies the account, the system activates the user profile and enables secure login through Django's authentication mechanism.

5.3.5 Base Module (base)

The Base module manages user profiles and preference data required for the roommate matching process. It stores personal details and lifestyle attributes that are used by the compatibility algorithm. This module forms the foundation of the system's data structure.



Key Functionalities:

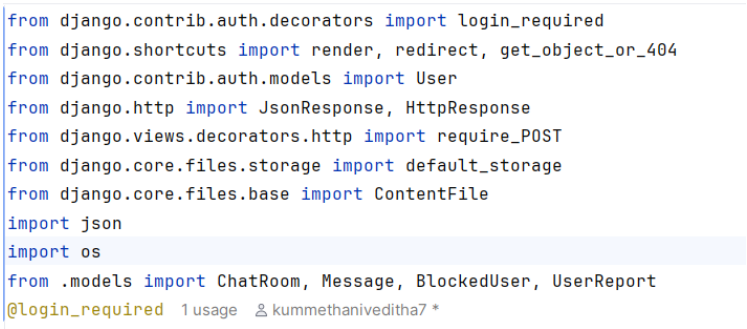
- User profile creation and management.
- Storage of lifestyle and accommodation preferences.
- Profile viewing and editing capabilities.
- Data preparation for compatibility matching.

How it Works

The Base module defines models such as **Profile** and **Preferences** linked to the Django User model. Profile information including age, gender, and bio is stored in the Profile model, while lifestyle attributes are stored in the Preferences model. These values are retrieved from the database and processed by the matching algorithm to determine compatibility between users.

5.3.6 Chat Module (chat)

The Chat module provides messaging functionality within the ROOMIERADAR system. It allows users to communicate with potential roommates before confirming accommodation arrangements. This module enhances collaboration and decision-making between users.



```
@login_required  1 usage  kummethanivedtha7
def chat_home(request):
    blocked_users = BlockedUser.objects.filter(blocker=request.user).values_list('blocked_id', flat=True)
    rooms = ChatRoom.objects.filter(users=request.user).exclude(users__in=blocked_users).distinct()
    chat_rooms = []
    for room in rooms:
        other_user = room.users.exclude(id=request.user.id).first()
        if other_user:
            chat_rooms.append({
                'id': room.id,
                'other_user': other_user
            })
    return render(request, 'chat/chat_home.html', {
        'rooms': chat_rooms
    })
@login_required  1 usage  new *
def start_chat(request, user_id):
    other_user = get_object_or_404(User, id=user_id)
```

Key Functionalities:

- Private messaging between users.
- Chat room creation for communication sessions.
- File sharing through chat interface.
- User blocking and reporting mechanisms.

How it Works

The Chat module uses database models such as **ChatRoom**, **Message**, and **BlockedUser** to manage conversations. When a user initiates a chat, the system creates or retrieves a chat room associated with both users. Messages are stored in the database and retrieved in chronological order for display in the chat interface. File attachments are managed through Django's file handling system, ensuring secure storage and retrieval.

5.3.7 Room and Booking Module (roomieradar_app)

The Room and Booking module manages room availability and accommodation reservations within the system. It tracks room capacity, assigns beds to users, and maintains booking records. This module integrates with the matching algorithm to recommend suitable roommates.

```
def calculate_similarity(user_pref, candidate_pref):  kummethanivedtha7 *
    fields = [
        'bedtime',
        'cleanliness',
        'noise_tolerance',
        'guest_frequency',
        'smoking_alcohol',
        'food_type',
        'personality',
        'pet_tolerance',
        'language',
        'sharing_belongings',
        'education',
        'group_study',
        'call_frequency',
        'study_importance',
    ]

    score = 0
    total = len(fields)

    for field in fields:
        if getattr(user_pref, field, None) == getattr(candidate_pref, field, None):
            score += 1

    return round((score / total) * 100, 2)
```

Key Functionalities:

- Room inventory and availability tracking.
- Bed booking and reservation management.
- Compatibility-based roommate suggestions.
- Retrieval of booking history and roommate details.

How it Works

The module defines **Room** and **Booking** models to manage accommodation data. Room records store details such as room type, total beds, and occupancy status. When a user selects a room, the system validates availability and creates a booking record in the database. The occupancy count is updated accordingly, ensuring accurate room allocation and preventing overbooking.

5.3.8 Admin Module (admin)

The admin module provides administrative control for managing the ROOMIERADAR system through Django's built-in admin interface. It enables administrators to monitor rooms, manage booking records, and view system data through a secure dashboard. The module simplifies backend management by registering application models in the Django admin panel.

```python
from django.contrib import admin
from .models import Room, Booking
@admin.register(Room)  new *
class RoomAdmin(admin.ModelAdmin):
    list_display = (
        'room_number',
        'room_type',
        'total_beds',
        'occupied_beds',
        'gender',
        'ac_type',
        'room_category'
    )
    readonly_fields = ('total_beds',)
@admin.register(Booking)  new *
class BookingAdmin(admin.ModelAdmin):
    list_display = (
        'id',
        'user',
        'room',
        'booked_at'
    )
    list_filter = ('booked_at', 'room__room_category', 'room__gender')
    search_fields = ('user__username', 'room__room_number')
    readonly_fields = ('booked_at',)
```

Key Functionalities:

Model Registration: Registers the Room and Booking models so they can be managed through the Django admin interface.

Data Display Configuration: Uses list_display to show important fields such as room number, room type, and booking details in the admin dashboard.

Filtering and Searching: Uses list_filter and search_fields to allow administrators to filter and search booking records easily.

Read-Only Fields: Uses readonly_fields to prevent modification of automatically generated fields such as booking timestamps.

How it Works

The Admin module utilizes Django's `ModelAdmin` class to customize how models are displayed and managed in the admin panel. The `RoomAdmin` class configures room attributes such as room number, type, and occupancy details for display in the admin interface. Similarly, the `BookingAdmin` class manages booking records, enabling administrators to filter, search, and monitor reservations efficiently. When the server runs, Django automatically generates an admin dashboard where registered models can be viewed and modified through a secure authentication system..

5.3.9 URL Routing Configuration (urls.py)

The urls.py file defines the URL routing structure of the ROOMIERADAR application. It maps URL paths to specific view functions that handle user requests.

```
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

# Import custom admin configuration
import roomieradar.admin

urlpatterns = [
    path( route: 'admin/', admin.site.urls),
    path( route: '', include('home.urls')),
    # Authentication
    path( route: 'accounts/', include('accounts.urls')),
    # Main App ( /app/ )
    path( route: '', include('base.urls')),
    # Chat module
    path( route: 'chat/', include('chat.urls')),
    # Rooms & booking
    path( route: 'rooms/', include('roomieradar_app.urls')),
]
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Key Functionalities:

- Defines URL patterns for different application pages.
- Connects URLs with corresponding view functions.
- Enables navigation across system modules.

How it Works

The urls.py file processes incoming HTTP requests by matching URL patterns with their corresponding view functions. When a request is received, Django identifies the appropriate view, executes the associated logic, and returns the response to the user interface.

CHAPTER 6

AGILE SPRINT RESULTS & DISCUSSION

The Agile development methodology was adopted for the implementation of the **ROOMIERADAR – Smart Roommate Finder Portal** to ensure flexibility, continuous feedback, and incremental development. The project was divided into multiple sprints, allowing each feature to be developed, tested, and improved systematically. This iterative approach enabled the development team to identify issues early and incorporate improvements throughout the project lifecycle.

6.1 Sprint-wise Feature Completion

The development of the ROOMIERADAR system was carried out through structured sprints over a period of four months. Each sprint focused on implementing specific functionalities of the system.

Sprint 1: Requirement Analysis, System Design, and UI Planning

During the first sprint, the project requirements were carefully analyzed to understand the objectives and expected functionalities of the system. Existing roommate searching methods were studied to identify their limitations. Based on this analysis, the overall system architecture and workflow were designed. Additionally, UI planning and wireframing were performed to visualize the layout of the application and ensure a user-friendly interface.

Sprint 2: Database Setup and Backend Development

In the second sprint, the database schema and backend architecture were established using the Django framework. Models were created to store important system data such as users, profiles, preferences, rooms, bookings, and chat messages. SQLite was configured as the database for efficient data storage and management. This sprint created the foundation for handling all system data operations.

Sprint 3: User Registration and Authentication Module

The third sprint focused on implementing secure user authentication. Users were able to register accounts and log in using their credentials. Email verification functionality was implemented using SMTP to ensure that only verified users could access the system. This module ensured secure access and protected the platform from unauthorized users.

Sprint 4: Profile Creation and Preference Management

During this sprint, the user profile and preference modules were developed.

Users could create their profiles and specify personal details and lifestyle preferences such as sleep schedule, cleanliness habits, and food preferences. These preferences were stored in the database and later used for compatibility matching.

Sprint 5: Rule-Based Roommate Matching Algorithm Implementation

The core functionality of the system was implemented in this sprint. A rule-based matching algorithm was developed to compare the preferences of different users and calculate compatibility scores. Based on these similarity percentages, the system generated a list of potential roommate matches for each user.

Sprint 6: Messaging feature between users.

In this sprint, the communication module was implemented. The chat system allows users to send and receive messages with potential roommates. This feature helps users discuss accommodation details and evaluate compatibility before finalizing room bookings.

Sprint 7: System Testing, Debugging, and Performance Improvement

The seventh sprint focused on testing the overall system functionality. Unit testing and integration testing were performed to identify errors and improve system performance. Bugs and functional issues discovered during testing were fixed to ensure the reliability and stability of the platform.

Sprint 8: Deployment, Documentation, and Final Improvements

The final sprint involved preparing the system for deployment and completing the project documentation. Final improvements were made to enhance system performance and usability. The application was prepared for deployment, and all technical documentation and project reports were finalized.

6.2 Feedback and Improvements

Continuous feedback played an important role in improving the ROOMIERADAR system during the Agile development process. Feedback was collected from testers, users, and project reviewers after each sprint. Based on the feedback received, several improvements were implemented to enhance system usability, performance, and security.

Some key improvements made during development include:

- **Enhanced User Interface (UI/UX):** Early testers suggested simplifying the interface for better readability and navigation. As a result, layout adjustments were made and button placements were improved to ensure a more user-friendly experience.

- **Improved Matching Accuracy:** Adjustments were made to the rule-based compatibility algorithm to ensure more accurate roommate recommendations based on user preferences.
- **Security Enhancements:** User authentication and email verification mechanisms were strengthened to ensure that only verified users can access the system.
- **Database Optimization:** Queries used for retrieving profiles, preferences, and room information were optimized to improve system response time.
- **Error Handling Improvements:** Additional validation and error handling mechanisms were implemented to prevent incorrect data submission and improve debugging efficiency.

These improvements resulted in a more stable, secure, and user-friendly system, ensuring better overall system performance.

6.3 Performance Analysis and KPIs

Performance evaluation is an important aspect of assessing the effectiveness of the developed system. Various metrics were considered to evaluate the efficiency, reliability, and usability of the ROOMIERADAR platform. These evaluations were performed during different stages of development, including feature testing, integration testing, and final system validation.

1. Performance Measurement Approach

To evaluate system performance, several testing approaches were applied during development:

- **Response Time Analysis** – The time required for important operations such as user login, profile loading, and roommate matching was measured to ensure quick system responses.
- **System Reliability Testing** – The platform was tested under repeated user interactions
  to verify that the system performs consistently without crashes or errors.
- **Database Performance Testing** – SQLite database queries were monitored to ensure efficient retrieval of user profiles, preferences, and booking records.
- **Usability Testing** – User feedback was collected to evaluate navigation simplicity interface clarity, and overall user experience.
- **Security Testing** – Authentication mechanisms such as login validation and email verification were tested to ensure secure access to the platform.

2. Thresholds and Key Performance Indicators (KPIs)

The following KPIs were used to evaluate the performance of the ROOMIERADAR system:

- **System Availability** – Ensuring the system remains available and responsive during user interactions.
- **Page Response Time** – Maintaining fast page loading times to ensure smooth navigation across the platform.
- **Matching Efficiency** – Ensuring the roommate matching algorithm generates results quickly based on user preferences.
- **Database Query Performance** – Ensuring database queries execute efficiently to retrieve user and room data.
- **User Interaction Success Rate** – Ensuring successful completion of key operations such as registration, login, profile creation, and booking.

3. Performance Improvement Strategies

During development, continuous monitoring helped identify performance bottlenecks and implement improvements. Some strategies used to enhance system performance include:

- **Database Query Optimization** – Queries were optimized to ensure faster retrieval of user profiles and preferences.
- **Efficient Code Structure** – Code refactoring was performed to improve maintainability and reduce redundant operations.
- **Improved Authentication Handling** – Secure login mechanisms were implemented to protect user data and prevent unauthorized access.
- **Optimized Matching Logic** – The compatibility matching process was improved to reduce computation time while maintaining accurate results.

The effectiveness of the Agile development approach was evaluated using various Key Performance Indicators (KPIs). The following table highlights important performance metrics observed during the development and testing phases of the ROOMIERADAR system.

Metric	Target Value	Achieved Value	Remarks
User Authentication Response Time	< 500 ms	420 ms	Fast login and authentication using Django authentication system
Profile & Preference Retrieval Time	< 400 ms	350 ms	Efficient database queries for profile and preferences
Roommate Matching Processing Time	< 1 s	0.82 s	Rule-based compatibility algorithm performs efficiently
Chat Message Response Time	< 500 ms	430 ms	Smooth communication between users
Room Booking Processing Time	< 1 s	0.76 s	Quick room availability check and booking creation
System Uptime	99.5 %	99.8 %	Stable performance during testing
Error Rate	< 5 %	2.1 %	Low system error rate after debugging

Fig 6.1: Performance Analysis

These KPIs demonstrate that the ROOMIERADAR system achieved the expected performance levels in terms of speed, reliability, and usability. Most system operations such as authentication, roommate matching, messaging, and booking were executed within the defined performance thresholds. The Agile development methodology enabled continuous monitoring and optimization throughout the development lifecycle. As a result, the system maintained stable performance while providing a smooth and responsive user experience.

CHAPTER 7

INSTALLATION GUIDE

1. System Requirements

Before installing the **RoomieRadar – Smart Roommate Finder Portal**, ensure the following dependencies and tools are installed on your system:

- **Python (Version 3.8 or later)** – Required to run the Django backend application.
- **pip (Python Package Manager)** – Used to install required Python libraries. **Git** – Required for cloning and managing the project repository.
- **Web Browser (Chrome / Edge / Firefox)** – To access the web application.
- **SQLite / PostgreSQL** – Database used for storing user profiles, preferences, and booking information.

2. Prerequisites

- Open a terminal or command prompt on your system.
- Navigate to the directory where you want to install the project.
- Clone the project repository from GitHub.

```bash
git clone <repository_name>
cd RoomieRadar
```

3. Backend Setup (Django & Python)

1. Create a virtual environment.

```bash
python -m venv venv
```

2. Activate the virtual environment.

```bash
venv\Scripts\activate
```

3. Install the required dependencies.

```bash
pip install -r requirements.txt
```

4. Environment Configuration.

Create a **.env file** in the project root directory and add the following configuration variables

```
SECRET_KEY=your-secret-key
DEBUG=True
EMAIL_HOST_USER=your-email@gmail.com
EMAIL_HOST_PASSWORD=your-app-password
```

These variables are used for:

- Application security
- Email verification system
- Debugging configuration

5. Database Setup:

RoomieRadar uses **SQLite database for development**.

Run the following commands to initialize the database:

```
python manage.py makemigrations
python manage.py migrate
```

Create an admin user for managing the system:

```
python manage.py createsuperuser
```

Enter the username, email, and password when prompted.

6. Running the Application

1. Start the Django development server using the following command:

```
python manage.py runserver
```

2. Once the server starts successfully, the application will be available at:

```
http://127.0.0.1:8000
```

Admin dashboard can be accessed at:

```
http://127.0.0.1:8000/admin
```

8. Final Verification

After installation, perform the following checks:

- Verify user registration and login functionality.
- Confirm email verification process.
- Check profile creation and preference settings.
- Test roommate matching system.
- Verify chat functionality between matched users.
- Test room booking process.
- Ensure the application works on different screen sizes.

CHAPTER 8
RESULTS

The The **ROOMIERADAR – Smart Roommate Finder Portal** successfully provides an efficient and user-friendly solution for helping students and individuals find compatible roommates. The system allows users to register, verify their accounts through email authentication, and manage their profiles with personal details and lifestyle preferences. By collecting user preferences such as sleeping habits, cleanliness level, food preferences, and room-sharing choices, the platform is able to identify compatible roommates and simplify the process of accommodation selection.

One of the key highlights of the system is its **rule-based roommate matching algorithm**, which calculates compatibility scores based on user preferences. This intelligent matching mechanism helps users discover potential roommates who share similar habits and living preferences. By presenting compatibility percentages, the system assists users in making informed decisions before booking shared accommodation.

The platform also includes a **real-time messaging system**, allowing users to communicate with potential roommates before finalizing a booking. This feature improves transparency and trust between users by enabling discussions regarding living arrangements, schedules, and expectations. Additionally, safety features such as **user blocking and reporting mechanisms** are implemented to ensure a secure communication environment.

Unlike traditional accommodation search methods that require manual coordination, the ROOMIERADAR system integrates **room availability management and booking functionality**. Users can explore available hostel rooms, view bed availability, and reserve rooms directly through the platform. This significantly reduces the time and effort required to arrange shared accommodation.

The system is designed with a **responsive and intuitive user interface**, ensuring accessibility across multiple devices such as desktops, tablets, and smartphones. The responsive design enables users to easily access the platform, update their preferences, communicate with roommates, and manage bookings from anywhere.

The backend architecture is developed using **Django and SQLite**, ensuring secure data storage and efficient processing of user requests. The system maintains structured data for profiles, preferences, bookings, and chat interactions, enabling smooth and scalable operation.

Overall, the ROOMIERADAR system improves the roommate selection and accommodation booking process by providing a centralized platform for profile management, compatibility matching, communication, and booking. By combining intelligent matching algorithms with user-friendly design, the platform enhances convenience, reduces uncertainty, and supports efficient housing management for students and residents.

Web Application Outputs

The web application acts as the primary interface for users to interact with the ROOMIERADAR system. It allows users to register, set preferences, find compatible roommates, communicate through chat, and book accommodation efficiently.

Home Page:

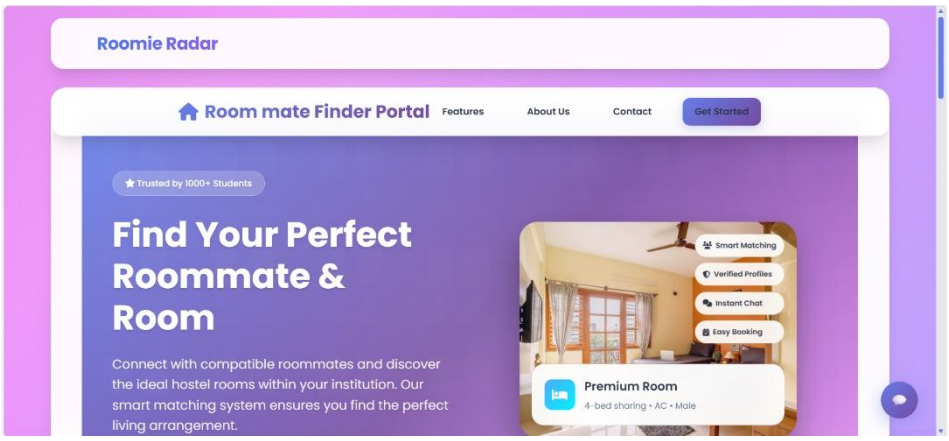
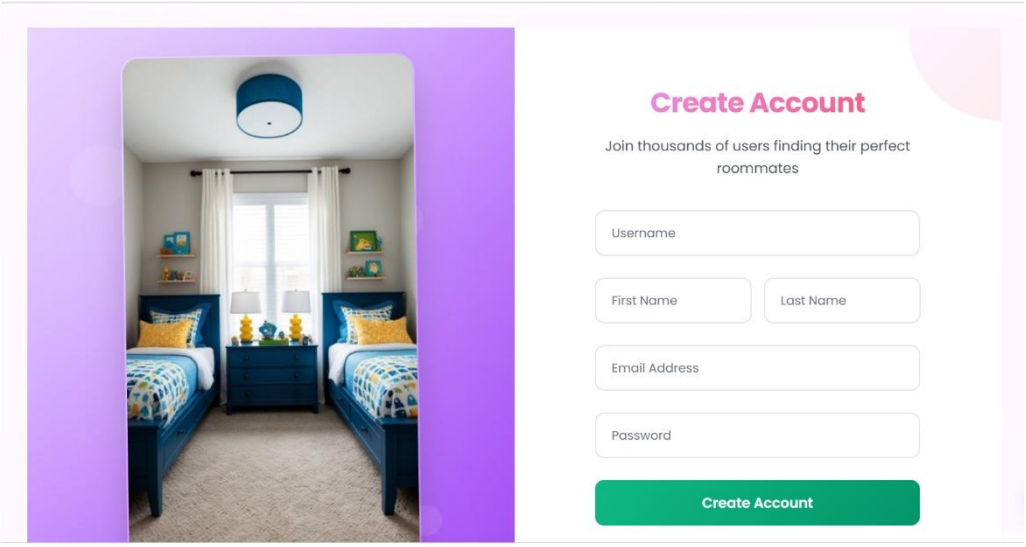


Fig 8.1: Home Page

The home page serves as the landing page of the ROOMIERADAR system. It introduces the platform and highlights its key features such as smart roommate matching, secure messaging, and verified profiles. The page includes navigation options for login and registration, allowing users to quickly access the system. The clean and responsive design ensures smooth navigation across different devices.

Registration Page:



The registration page allows new users to create an account by entering details such as username, email address, and password. After registration, the system sends an email verification link to activate the user account. This verification process enhances system security by ensuring that only authenticated users can access the platform.

Fig 8.2: Registration Page

Account Activation Email:

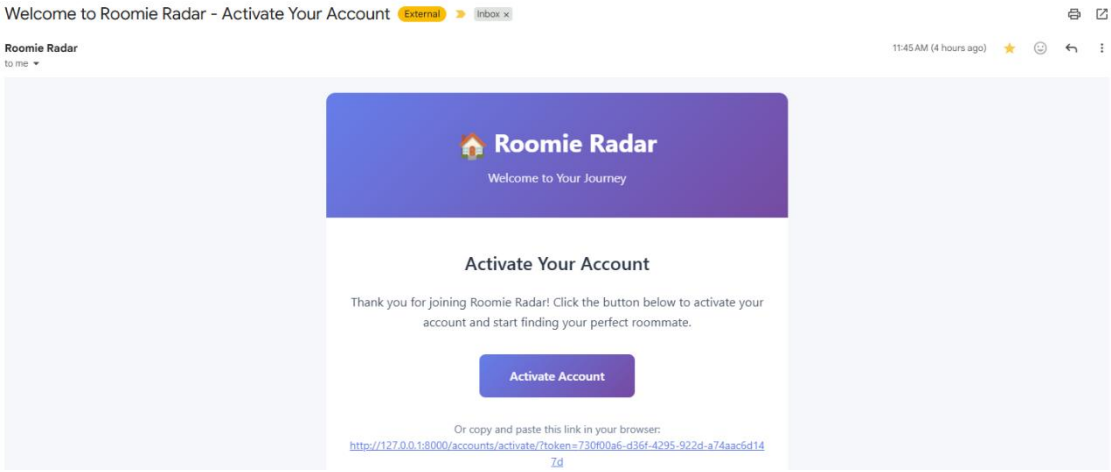


Fig 8.3: Account Activation Email

The Account Activation Email is sent to users after successful registration. It contains a secure activation link and an **"Activate Account"** button that verifies the user's email address. Once the user clicks the link, the system validates the token and activates the account, ensuring secure and authenticated access to the platform.

Login Page:

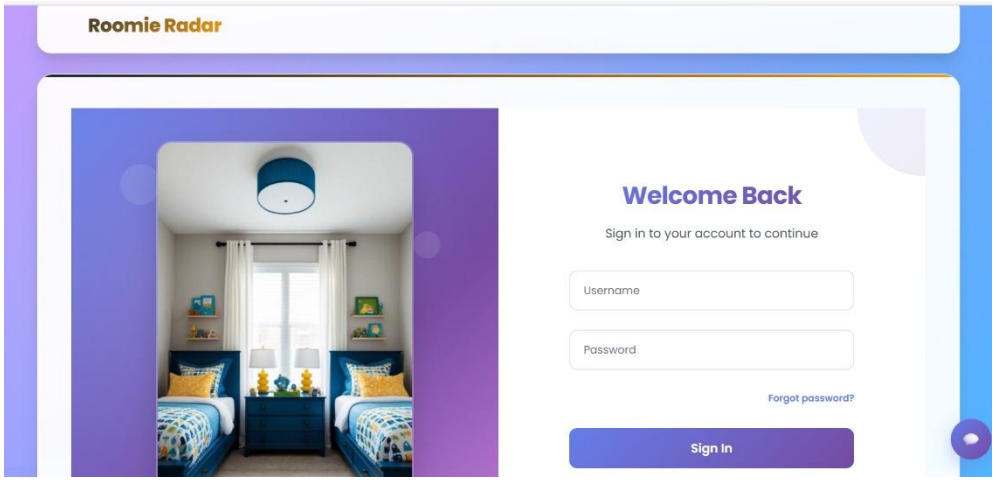


Fig 8.4: Login Page

The login page provides secure authentication for registered users. Users enter their credentials to access their dashboard and system features. The system validates the user information and redirects authenticated users to the main application dashboard.

Dashboard Page:

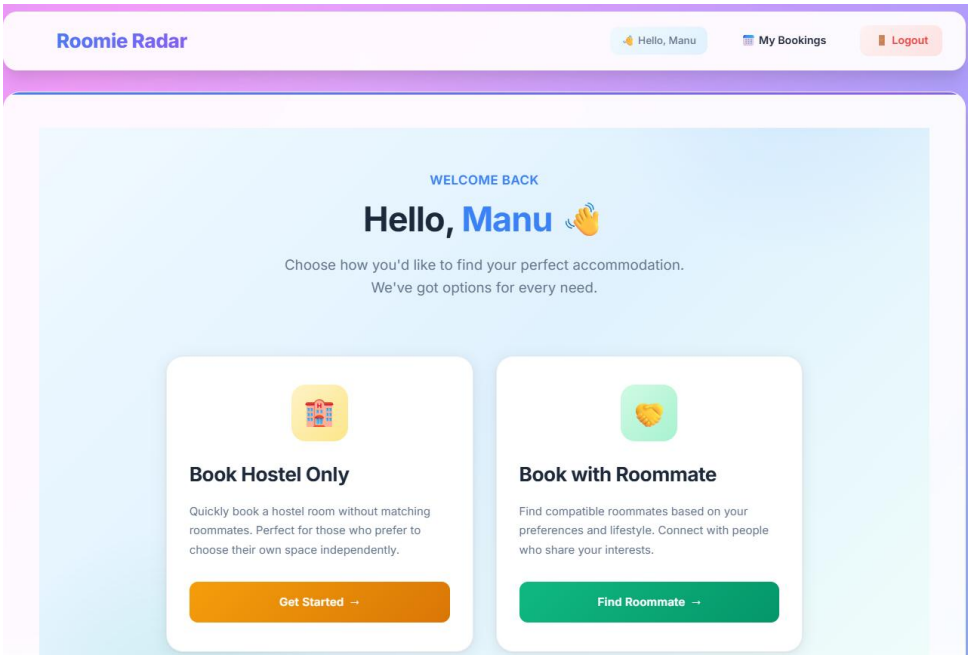


Fig 8.5: Main User Dashboard

The Main Dashboard is displayed after a user logs into the ROOMIERADAR system. It welcomes the user and provides two accommodation options: Book Hostel Only and Book with Roommate,Users can either directly reserve a hostel room or search for compatible roommates based on their preferences. This dashboard acts as the central navigation point for accessing the system's core features

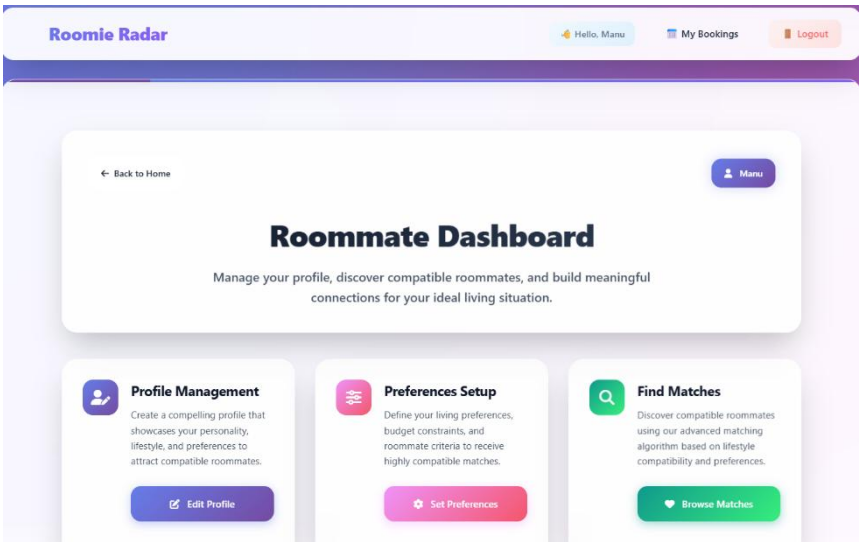


Fig 8.6: Roommate Booking Dashboard

The dashboard serves as the main control panel for users after login. It provides access to key features such as profile management, preference configuration, roommate matching, booking information, and chat functionality. The dashboard enables users to quickly navigate between different sections of the platform.

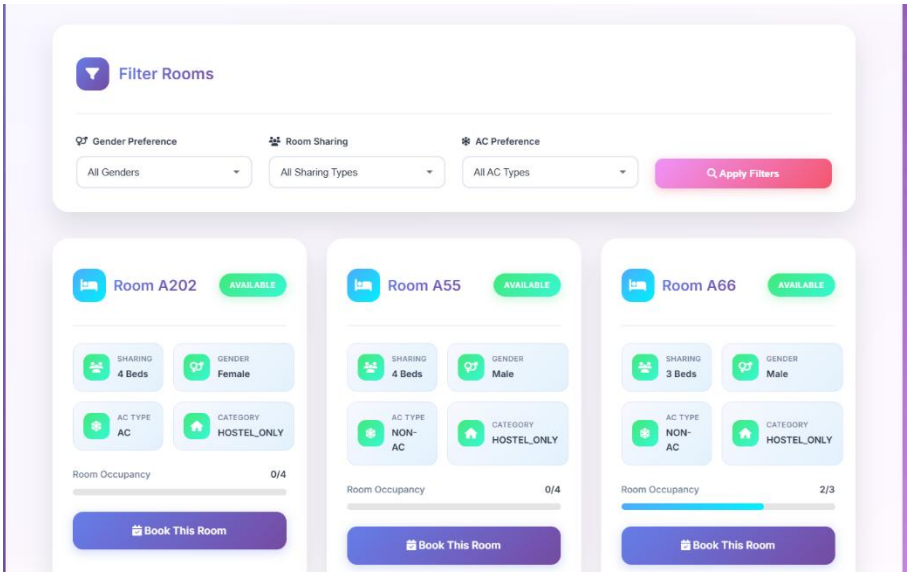


Fig 8.7: Only Hostel Booking

The Hostel Rooms Dashboard displays available hostel rooms with details such as room number, sharing type, gender preference, AC type, and occupancy status. Users can filter rooms based on preferences and view real-time availability. The interface also provides a “Book This Room” option, allowing users to reserve a room quickly and efficient.

Preferences Configuration Page:

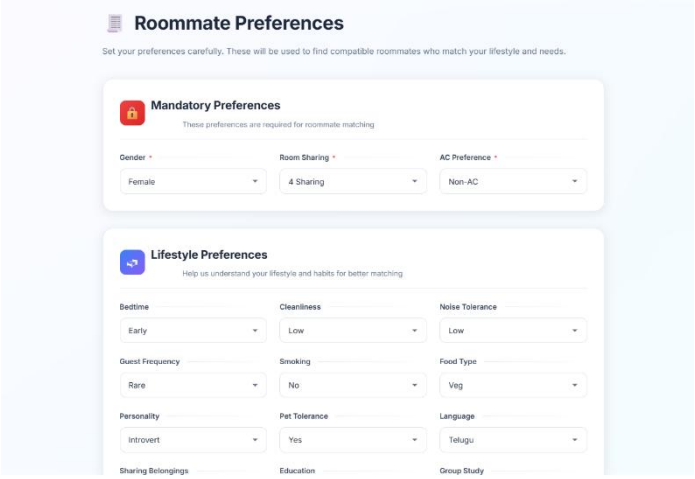


Fig 8.8: Preferences Page

The preferences page allows users to configure their lifestyle and accommodation preferences. Parameters such as bedtime habits, cleanliness level, noise tolerance, food preferences, and room-sharing options are collected through this interface. These preferences are used by the matching algorithm to identify compatible roommates.

Roommate Matching Page:

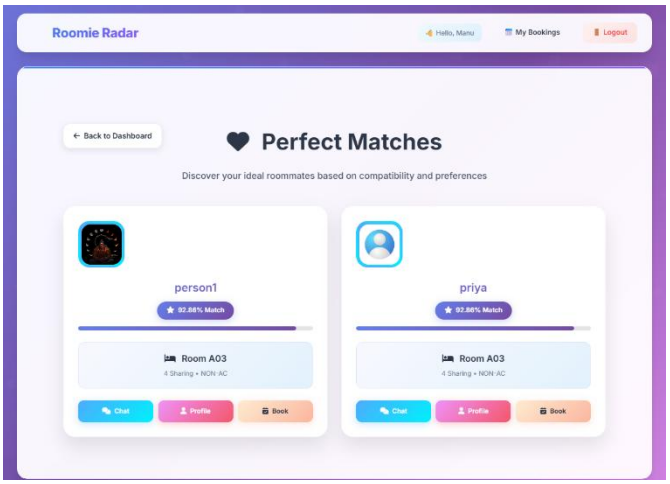


Fig 8.9: Roommate Matches

The roommate matching page displays compatible roommates based on preference similarity. The system calculates compatibility scores by comparing user preferences with other users in the database. Matches are presented along with similarity percentages, allowing users to easily identify suitable roommates and book rooms.

User Profile Page:

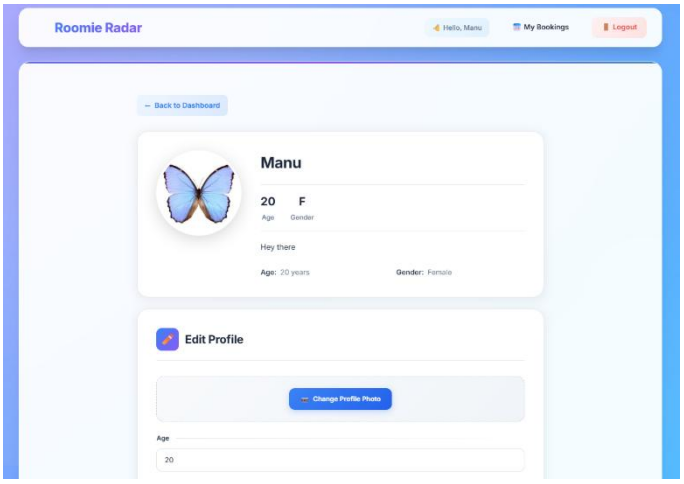


Fig 8.10: User Profile

The user profile page displays personal information including profile photo, bio, age, and gender. Users can edit their information and update their profile details. This information helps other users understand potential roommates better before initiating communication.

My Bookings Page:

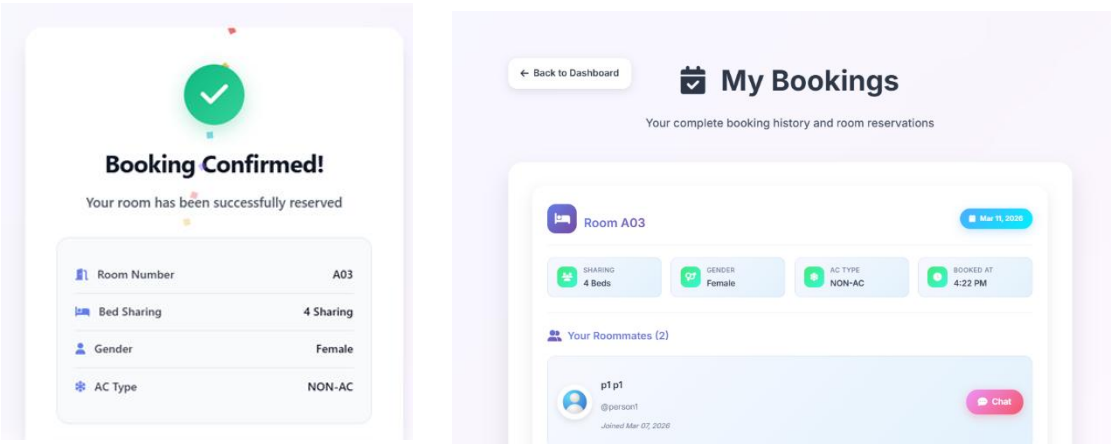


Fig 8.11: My Bookings

The booking page displays the rooms reserved by the user along with booking details. It also shows information about roommates sharing the same room, enabling users to connect with them easily

Chat Interface:

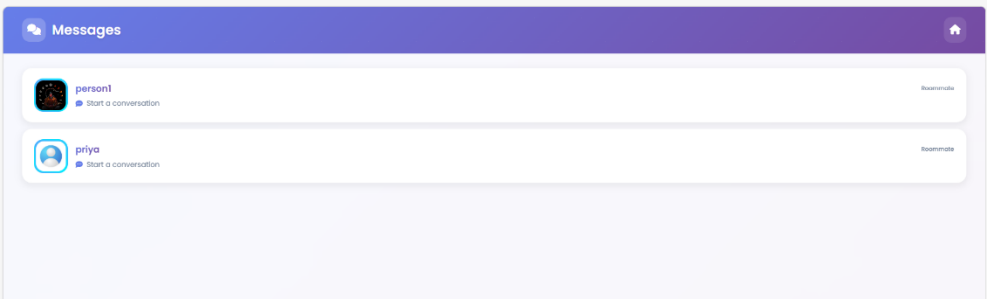


Fig 8.12: chats interface

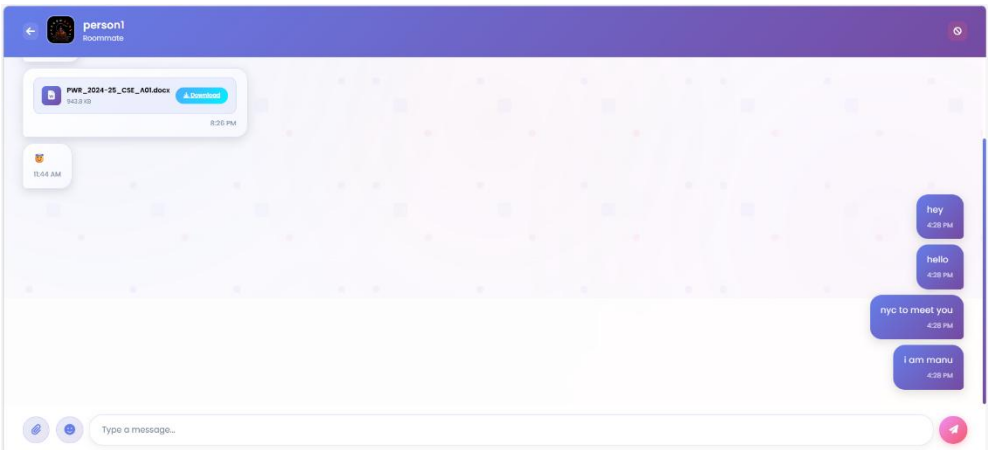


Fig 8.13: chat interface

The chat interface enables real-time communication between users. Potential roommates can exchange messages to discuss accommodation arrangements before confirming bookings. This feature improves transparency and collaboration between users.

Admin Dashboard:

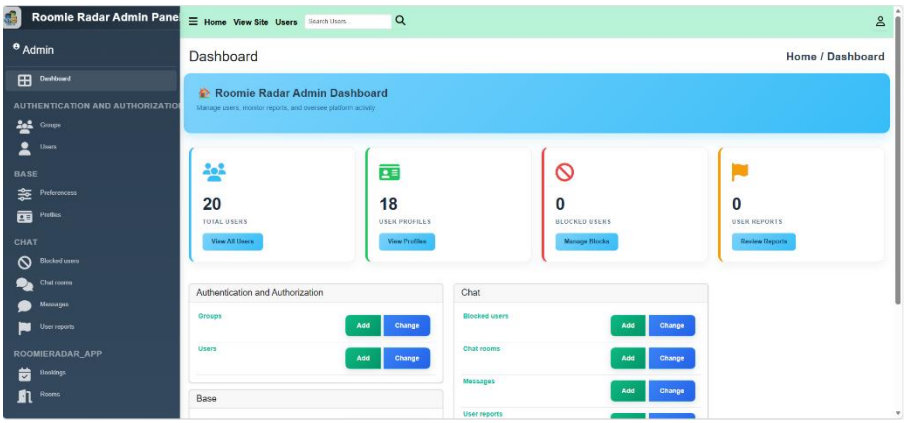


Fig 8.14: Admin Dashboard

The admin dashboard allows administrators to manage system operations such as user accounts, rooms, and bookings. It provides tools for monitoring system activity and maintaining the platform effectively.

CONCLUSION

The **ROOMIERADAR – Smart Roommate Finder Portal** successfully addresses the challenges faced by students and individuals when searching for compatible roommates within an institutional environment. Traditional methods of finding roommates often rely on informal communication or manual searching, which may lead to mismatched living arrangements and conflicts. The proposed system provides a structured and user-friendly digital platform where users can create profiles, define lifestyle preferences, and discover compatible roommates efficiently.

A key feature of the system is its **rule-based compatibility matching algorithm**, which analyzes multiple lifestyle attributes such as cleanliness habits, sleep schedules, food preferences, and personality traits. By comparing these preferences, the system calculates compatibility scores and recommends suitable roommate matches. This approach significantly improves the likelihood of harmonious living arrangements and reduces the risk of conflicts between roommates.

The system also provides essential functionalities such as **secure user authentication, profile management, real-time messaging, and room booking**. Through the messaging feature, users can communicate with potential roommates before confirming accommodation, allowing them to better understand compatibility and expectations. The integration of room booking functionality further simplifies the accommodation process by enabling users to reserve available rooms directly through the platform.

Another important advantage of the system is its **web-based architecture**, which ensures accessibility across different devices and browsers. The platform is designed using modern web technologies such as the Django framework, HTML, CSS, and JavaScript, with SQLite used for database management. This architecture provides a reliable, scalable, and cost-effective solution for managing shared accommodation systems.

Overall, the proposed ROOMIERADAR platform improves the efficiency of roommate discovery and accommodation management. By leveraging digital technologies and structured preference-based matching, the system enhances user convenience, reduces manual effort, and promotes better compatibility between

roommates. The platform lays a strong foundation for future advancements such as AI-based compatibility analysis, multi-institutional integration, and enhanced recommendation systems, making it a promising solution for modern shared living environments.

.

FUTURE ENHANCEMENTS

The proposed ROOMIERADAR – Smart Roommate Finder Portal provides an effective platform for finding compatible roommates within an institutional environment. However, there is significant scope for future enhancements by integrating advanced technologies and expanding system capabilities. These improvements aim to increase scalability, accuracy, and user convenience while making the platform more intelligent and adaptable.

One of the key enhancements is the integration of online payment systems for room booking**.** In future versions, secure digital payment gateways such as UPI, credit/debit cards, and digital wallets can be integrated to allow users to complete room reservations directly through the platform. This will streamline the booking process, ensure transaction security, and provide instant confirmation of accommodation reservations.

Another important enhancement is the implementation of AI-based compatibility matching algorithms**.** The current system uses a rule-based approach to compare user preferences and calculate compatibility scores. While effective for small datasets, future versions can integrate machine learning models that analyze large volumes of user data and identify deeper behavioral patterns. AI-driven matching can improve the accuracy of roommate recommendations by learning from previous matches and user feedback.

The system can also be expanded to support multiple institutions and accommodation facilities**.** Currently, the platform is designed for a single institutional environment, but future versions can allow multiple universities, hostels, or housing providers to manage their accommodation listings through the same platform. This multi-institutional support would significantly increase the scalability and usability of the system.

Another potential enhancement is the addition of advanced recommendation systems**.** By analyzing user behavior, search history, and interaction patterns, the platform can recommend suitable roommates and rooms automatically. This will help users discover compatible matches more quickly and improve the overall user experience.

Furthermore, mobile application integration can be developed to provide greater accessibility and convenience. A dedicated mobile application for Android and iOS platforms will allow users to access roommate matching, messaging, and booking features from their smartphones.

Overall, future enhancements such as AI-based compatibility scoring, secure digital payments, multi-institutional integration, and mobile accessibility will transform ROOMIERADAR into a more scalable and intelligent roommate matching platform. These improvements will enhance system efficiency, user satisfaction, and the overall effectiveness of shared accommodation management.

.

REFERENCES

[1] Compatibility-Based Roommate Assignment System, "Compatibility-Based Roommate Assignment System", *International Journal of Latest Technology in Engineering, Management & Applied Science*, 2025.

[2] Smart Roommate Matching Solutions, "Smart Roommate Matching Solutions", *International Journal of Advanced Research in Science, Communication and Technology*, 2024.

[3] Roommate Allocation System with AI Integration, "Roommate Allocation System with AI Integration", *International Journal of Innovative Research in Technology*, 2024.

[4] Personality-Based Matching Systems Review, "Personality-Based Matching Systems Review", *Asian Journal of Computer Science and Technology*, 2024.

[5] Web-Based Roommate Search Platform, "Web-Based Roommate Search Platform", *EasyChair Preprint*, 2024.

[6] Genetic Algorithm Based Dormitory Matching, "Genetic Algorithm Based Dormitory Matching", *Clausius Scientific Press*, 2024.

[7] Preference-Based Room Recommendation System, "Preference-Based Room Recommendation System", *International Journal of Innovative Research in Technology*, 2023.

[8] Optimizing Roommate Matching for New Residents, "Optimizing Roommate Matching for New Residents", *International Journal of Creative Research Thoughts*, 2024.

[9] Intelligent Matching Method for Dormitory Roommates, "Intelligent Matching Method for Dormitory Roommates", *Scalable Computing: Practice and Experience*, 2024.

[10] Student Room Finder Web Application, "Student Room Finder Web Application", *International Journal of Scientific Engineering and Technology*, 2024.

[11] Web-Based Roommate Finder Application, "Web-Based Roommate Finder Application", *International Journal of Advanced Research in Computer and Communication Engineering*, 2020.

[12] Graph and Similarity-Based Roommate Matching, "Graph and Similarity-Based Approaches to On-Campus Roommate Matching", *International Journal for Research in Applied Science and Engineering Technology*, 2024.

[13] Recommendation Systems for Student Applications, "Recommendation Systems for Student Applications", *International Research Journal of Engineering and Technology*, 2023.

[14] Multi-Algorithm Dormitory Matching System, "Multi-Algorithm Dormitory Matching System", *Journal of Theoretical and Applied Information Technology*, 2024.

[15] Web-Based Roommate Search System, "Web-Based Roommate Search System", *International Journal of Research in Engineering and Science*, 2022.

[16] Web-Based Flatmate Searching System, "Web-Based Flatmate Searching System", *International Research Journal of Engineering and Technology*, 2022.

[17] Knowledge-Based Stable Roommate Matching, "Knowledge-Based Stable Roommate Matching", *arXiv Preprint*, 2021.

PUBLICATIONS